

Fase 1: Geração e Validação da Infraestrutura Base (SoC RISC-V)

A primeira etapa consiste em estabelecer a plataforma de hardware que servirá de alicerce para o sistema operacional e para o acelerador. O desenvolvimento de um processador do zero é inviável para o escopo e prazo, portanto, adotar-se-á um gerador de *System-on-Chip* (SoC).

- **Definição do Core RISC-V:** O núcleo escolhido deve possuir suporte nativo à execução de sistemas operacionais complexos como o Linux. Isso exige a seleção de uma variante da arquitetura RISC-V que contemple a extensão **A** (*Atomic Instructions*) e uma **MMU** (*Memory Management Unit*) completa para paginação de memória virtual e suporte aos modos *Machine*, *Supervisor* e *User* (ex: perfil RV32IMA ou superior). O núcleo *VexRiscv* (variante Linux) é altamente recomendado.
- **Utilização do Framework LiteX:** O sistema será construído utilizando o framework LiteX, baseado em Python. Este framework automatiza a criação do barramento interno (tipicamente Wishbone), a instanciação do núcleo RISC-V, do controlador de memória RAM, das memórias ROM de boot e dos periféricos básicos de entrada e saída (UART para o terminal Linux, temporizadores).
- **Síntese e Validação Física:** O código gerado (RTL) será exportado para a ferramenta de síntese do FPGA alvo (ex: Vivado para Xilinx ou Quartus para Intel). O objetivo primordial desta fase é gerar o *bitstream*, gravar no FPGA e atestar o funcionamento do hardware executando um firmware *bare-metal* simples (ex: teste de eco via terminal serial UART ou controle de LEDs via registradores).

Fase 2: Projeto Lógico da NPU Ternária (Acelerador *Multiplierless*)

Esta fase concentra a principal inovação do hardware: o desenvolvimento da Unidade de Processamento Neural Ternária (TNN). Diferente de aceleradores convencionais, esta NPU não utilizará blocos de multiplicação matemática complexa (DSP *slices*).

- **Substituição do MAC (*Multiply-Accumulate*):** Como os pesos da rede estão restritos aos valores $\{-1, 0, 1\}$, a operação matricial deve ser convertida para uma arquitetura baseada estritamente em somadores, subtratores e multiplexadores.
 - Se o peso de entrada for **00** (representando 0), a entrada de dados é descartada (soma 0).
 - Se o peso for **01** (representando 1), a entrada de dados é somada ao acumulador.
 - Se o peso for **11** (representando -1), a entrada de dados sofre complemento de 2 (inversão de sinal) e é somada ao acumulador (ou seja, é subtraída).
- **Decodificação e Desempacotamento de Dados (*Unpacking*):** A interface da NPU receberá palavras de 32 bits (ou 64 bits) do barramento. O módulo em linguagem de descrição de hardware (Verilog/VHDL) deve extrair (*slice*) os 16 pesos de 2 bits contidos nessa única palavra simultaneamente, alimentando uma matriz de somadores em paralelo para maximizar o rendimento (*throughput*).

- **Interface de Barramento e Mapeamento de Memória (MMIO):** A NPU deve ser encapsulada com uma interface de comunicação escrava padrão, como **Wishbone** ou **AXI4-Lite**. É necessário projetar um mapa de registradores acessíveis pela CPU:
 - *Registrador de Controle:* Para iniciar (Start) ou resetar a NPU.
 - *Registrador de Status:* Para indicar que o processamento terminou (Done/Ready).
 - *Registradores de Dados (Entrada e Saída):* Endereços físicos por onde o modelo receberá as amostras, os pesos e disponibilizará o resultado final.

Fase 3: Integração SoC-NPU e Simulação RTL

Antes da síntese final para o hardware físico, o módulo da NPU deve ser rigorosamente testado em ambiente de simulação para evitar ciclos longos de compilação no FPGA.

- **Acoplamento ao SoC:** O módulo Verilog da NPU será instanciado no script Python do LiteX, sendo acoplado como um periférico escravo no barramento principal do sistema, recebendo um endereço base físico fixo na memória.
- **Simulação via Verilator/ModelSim:** Utilizando ferramentas de simulação RTL, deve-se escrever *testbenches* para injetar sinais virtuais no barramento, simulando operações de escrita e leitura nos endereços físicos da NPU.
- **Validação via Software Bare-Metal:** Ainda em simulação, será escrito um pequeno programa em linguagem C (sem sistema operacional) que escreve matrizes fictícias nos endereços mapeados da NPU e verifica se o resultado acumulado lido no registrador de saída está matematicamente correto.

Fase 4: Síntese Física, Documentação e Extração de Métricas

A etapa final do desenvolvimento de hardware consiste em materializar o sistema completo, avaliar seu custo computacional e fornecer a interface abstrata para as equipes de software.

- **Fechamento de Timing (*Timing Closure*):** O projeto integrado (RISC-V + NPU Ternária) deve ser sintetizado no FPGA, garantindo que não ocorram violações de temporização (*slack* negativo) entre os registradores.
- **Análise de Utilização de Recursos:** Documentar o consumo de elementos lógicos do FPGA gerado pelos relatórios da ferramenta de síntese. A principal métrica para o artigo será demonstrar o baixo ou nulo uso de blocos multiplicadores (DSPs) e o consumo de *Look-Up Tables* (LUTs) e *Flip-Flops* (FFs), comprovando a eficiência de área da NPU Ternária.
- **Entrega do Mapa de Memória:** Elaboração de um documento técnico especificando o Endereço Base da NPU no hardware físico e os *offsets* de cada registrador de controle e dados. Este documento é o requisito fundamental para que o responsável pela *Device Tree* (Fase 2 do Software) e o desenvolvedor do Driver do Kernel (Fase 3 do Software) possam iniciar a comunicação com o periférico físico.